

UNIVERSITY OF MICHIGAN
Department of Electrical Engineering and Computer Science
EECS 445 — Introduction to Machine Learning
Fall 2025

Homework 2, Issued: Wed. 09/24 Due: Wed. 10/08 10pm ET

Submission: Please upload your completed assignment to Gradescope. Your submission can be either handwritten or typed. Assign all pages (including pages containing code) to their respective questions. Incorrectly assigned pages will not be graded.

Environment Setup for HW2: You should already have Anaconda installed for Project 1. Run the following command to create a conda environment named 445.hw2: `conda create --name 445.hw2 python=3.13.3 --file requirements.txt -c pytorch`

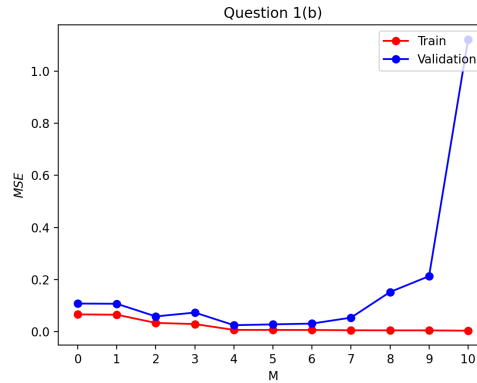
1 Linear Regression Optimization Method (8 pts)

For this question, you are provided datasets `q1_train.csv` and `q1_validation.csv` in the `q1_data` folder, along with a helper function in `q1_helper.py` to load the data. You will update the skeleton code in `q1_linear_regression.py` according to the instructions. You will investigate the problem of overfitting in linear regression, particularly as we increase the complexity of the model by increasing the polynomial degree M .

- (a) (2 pts) First, implement the function `generate_polynomial_features(X, M)` according to the specification in the skeleton code `q1_linear_regression.py`. This function transforms each example data point $x^{(i)}$ into an $M + 1$ dimensional feature vector, $\phi(x^{(i)})$. **Include a screenshot (or equivalent) of your implemented function as your solution to this question.**
- (b) We will investigate the problem of overfitting with linear regression. We expect to observe overfitting as we increase the degree of the polynomial, M . We evaluate solutions using mean squared error (MSE), defined as

$$MSE = \frac{1}{n} \sum_{i=1}^n (\bar{\theta} \cdot \phi(\bar{x}^{(i)}) - \bar{y}^{(i)})^2$$

- (i) (1 pt) Using the given functions `calculate_MS_Error(X, y, theta)` and `closed_form_optimization(X, y, reg_param=0)` in `q1_linear_regression.py`, find the $\bar{\theta}$ that minimizes the mean squared error for an M^{th} degree polynomial (for $M = 0 \dots 10$) on the training data in `q1_train.csv`. Using your values of $\bar{\theta}$, calculate the mean squared error, for each setting of M , on the training data and on the validation data in `q1_validation.csv` (separately). Plot MSE against M for both training data and validation data (in the same graph) and include it in your solution write-up. Your graph should look similar to the following:



- (ii) (1 pt) Which degree polynomial would you say best fits the data? Is there any evidence of underfitting / overfitting? Use your generated chart to defend your answer.

(c) Now, you will explore the role of regularization.

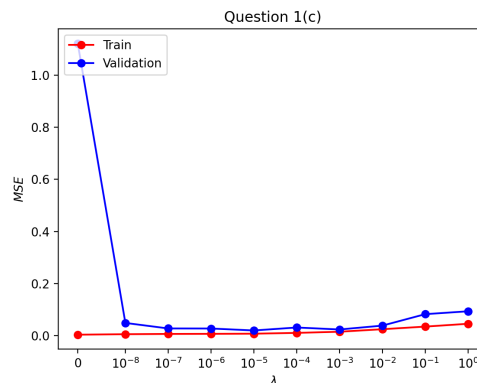
- (i) (2 pts) Modify the given implementation of `closed_form_optimization(X, y, reg_param=0)` to incorporate L2-regularization. Specifically, use the closed form solution associated with the following regularized objective function:

$$\lambda \frac{\|\bar{\theta}\|^2}{2} + \frac{1}{n} \sum_{i=1}^N \frac{(\bar{\theta} \cdot \phi(x^{(i)}) - y^{(i)})^2}{2}$$

Include a screenshot (or equivalent) of your implemented functions as your solution to this question.

- (ii) (1 pt) Use your function from part (c.i) to find the $\bar{\theta}$ that minimizes the regularized objective function for a tenth degree polynomial ($M = 10$) given regularization factor λ (for $\lambda \in \{0, 10^{-8}, 10^{-7}, \dots, 10^{-1}, 10^0\}$) for the training data specified in `q1_train.csv`. Now use these coefficients to calculate the mean squared error on both the training data and validation data as a function of λ and plot MSE against $\lambda \in \{0, 10^{-8}, 10^{-7}, \dots, 10^{-1}, 10^0\}$ and include it in your solution write-up.

Your plot should look similar to the following:



- (iii) (1 pt) Which value of λ appears to work the best? Provide justification.

2 Hyperparameter Selection (4 pts)

Nathan and Frederick have collected data on the number of pull requests each student submits per week for the 350 students enrolled in EECS 445 in Fall 2025. For each student, they also recorded age, gender, number of coding side projects, hours spent debugging per day, and average nightly sleep. They want to build a regularized linear regression model that predicts a student's weekly pull request activity based on these features. While Nathan and Frederick consider the same set of hyperparameters (regularization strength) in their experiments, they take different approaches to finding the optimal hyperparameter setting.

Both Nathan and Frederick agree to first randomly split the data once into a train and test set. And then:

- Nathan takes all of the training data and trains a model with each of the hyperparameter settings and then applies each model to the test data. In the end, he selects the hyperparameter that achieves the best performance on the test data, and reports this performance as his final test performance.
 - Frederick further randomly splits the training data into a subtrain and validation set. He trains a model with each of the hyperparameter settings on the subtrain and applies it to the validation set, measuring predictive performance. He repeats this process 5 times with different cross-validation folds. He then selects the hyperparameter setting that does best on average across all of the validation folds. Using this hyperparameter setting, he trains a model on all of the training data and then reports performance on the test set.
- (a) (2 pt) Compare the two proposed experimental setups. Which setup is likely to yield a more accurate estimate of how their models will perform on new data? Explain your reasoning.
- (b) (2 pt) Jimmy proposes a third alternative setup that all three agree is best. What did he propose?

3 Kernels (5 pts)

We discussed multiple methods to determine the validity of kernels in lecture. You will use three different methods to prove the validity of three different kernels in this problem.

- (a) (2 pts) Consider the kernel $K_1(\bar{u}, \bar{v}) = (\bar{u} \cdot \bar{v})$, $\bar{u}, \bar{v} \in \mathbb{R}^2$. Prove that K_1 is a valid kernel by constructing the Gram matrix G for an arbitrary dataset X and showing that it is symmetric and positive semi-definite.
- (b) (2 pts) Consider $K_2(\bar{u}, \bar{v}) = (1 + \bar{u} \cdot \bar{v})^2$ on $\mathbb{R}^2$. Find an explicit feature map ϕ such that $K_2(\bar{u}, \bar{v}) = \phi(\bar{u}) \cdot \phi(\bar{v})$.
- (c) (1 pt) Consider $K_3(\bar{u}, \bar{v}) = 4(\bar{u} \cdot \bar{v}) + (1 + \bar{u} \cdot \bar{v})^2$ on $\mathbb{R}^2$. Prove that K_3 is a valid kernel via “kernel algebra” (i.e., closure properties) and your results from parts (a) and (b). Cite the specific closure properties used in your solution.

4 Analyzing the RBF Kernel (4 pts)

Recall the Radial Basis Function (RBF) kernel from lecture:

$$K(\bar{\mu}, \bar{v}) = \exp(-\gamma \|\bar{\mu} - \bar{v}\|^2)$$

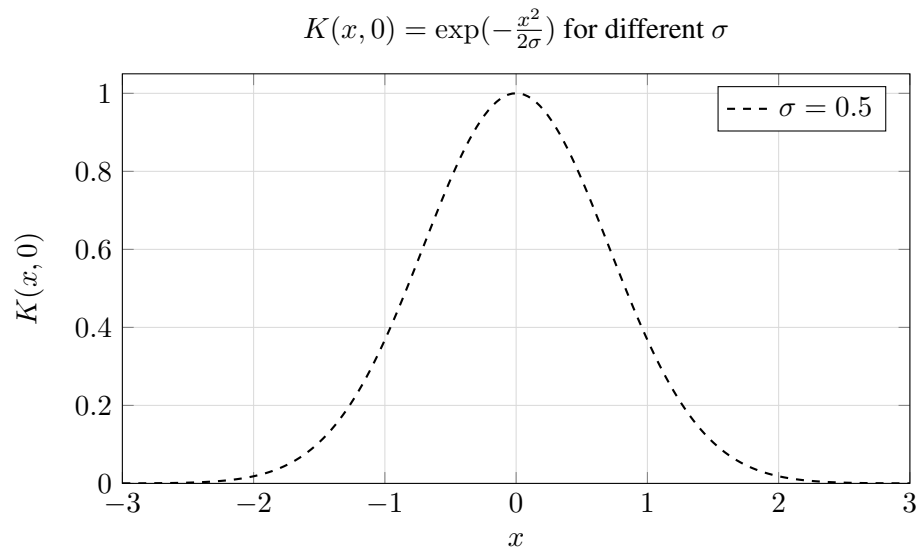
where γ is a positive hyperparameter. Oftentimes the hyperparameter for the RBF kernel is expressed as σ , where $\gamma = \frac{1}{2\sigma}$

- (a) (1pt) For each of the following pairs of points $u, v \in \mathbb{R}$, calculate the RBF kernel value when $\frac{1}{2\sigma} = \gamma = 1$:

$$u = 0, v = 0; \quad u = 0, v = 1; \quad u = 0, v = 2.$$

Briefly explain how the RBF kernel value relates to the distance between points.

- (b) (2pts) Consider a plot of $K(x, 0)$ versus x for different σ . We have provided you the plot for $\sigma = 0.5$. Sketch and describe two additional curves where $\sigma > 0.5$ and $\sigma < 0.5$. What does this imply about how different values of σ impact the kernel’s notion of similarity between two points?



- (c) (1pt) Explain how the notion of similarity between points affects the decision boundary in classification problems that use the RBF kernel. How would a classifier that uses an RBF kernel with a highly sensitive similarity metric compare to one with a less sensitive similarity metric?

5 Kernelizing Multiclass Perceptron Algorithm (8 pts)

Let's consider a variant of the perceptron algorithm, for a multi-class classification problem. The training set consists of n examples $\{\bar{x}^{(i)}, y^{(i)}\}_{i=1}^n$ where $\bar{x}^{(i)} \in \mathbb{R}^d$ and $y^{(i)} \in \{1, 2, 3, \dots, m\}$. To solve this problem, we will learn m different parameter vectors $\bar{\theta}_c$ for $c = 1, \dots, m$. The algorithm is as follows:

```

1 Initialization: for  $c = 1, \dots, m$ :  $\bar{\theta}_c^{(0)} = \bar{0}$ ,  $k = 0$ 
2 Repeat until convergence:
3   for  $i = 1, \dots, n$ :
4      $\hat{y}^{(k)} = \arg \max_y \bar{\theta}_y^{(k)} \cdot \bar{x}^{(i)}$ 
5     if  $\hat{y}^{(k)} \neq y^{(i)}$ :
6        $\bar{\theta}_{y^{(i)}}^{(k+1)} \leftarrow \bar{\theta}_{y^{(i)}}^{(k)} + \bar{x}^{(i)}$ 
7        $\bar{\theta}_{\hat{y}}^{(k+1)} \leftarrow \bar{\theta}_{\hat{y}}^{(k)} - \bar{x}^{(i)}$ 
8      $k \leftarrow k + 1$ 

```

After the algorithm has converged, new examples can be classified as $h(\bar{x}) = \arg \max_y \bar{\theta}_y \cdot \bar{x}$.

We would like to kernelize this algorithm. This can be done by mapping each example $\bar{x} \in \mathbb{R}^d$ into a new feature vector $\phi(\bar{x}) \in \mathbb{R}^{d'}$ (in many cases, $d' > d$) and reformulating the algorithm such that it only uses the kernel function $K(\bar{x}^{(i)}, \bar{x}^{(j)})$ corresponding to $\phi(\bar{x}^{(i)}) \cdot \phi(\bar{x}^{(j)})$. This will require a number of changes. In the algorithm above, we need to calculate $\bar{\theta}_y \cdot \bar{x}$. However, once we introduce a feature mapping $\phi: \mathbb{R}^d \rightarrow \mathbb{R}^{d'}$, $\bar{\theta}_y$ will lie in a d' -dimensional space, where d' may be very large. We want a new formulation that does not explicitly depend on $\bar{\theta}_y$.

- (2 pts) Write the prediction $\hat{y}$ in terms of $\alpha_{i,c}$ and the kernel K .
- (4 pts) Give the kernelized perceptron algorithm: initialize $\alpha_{i,c} = 0$. For each training example i , compute $\hat{y}$ using part (a). If $\hat{y} \neq y^{(i)}$, give the updates to $\alpha_{i,y^{(i)}}$ and $\alpha_{i,\hat{y}}$. Write down the pseudocode by filling in the blanks below.

```

1   Initialization: for  $c = 1, \dots, m$ :  $\bar{\theta}_c^{(0)} = \bar{0}$ ,  $k = 0$ 
2   Repeat until convergence:
3     for  $i = 1, \dots, n$ :
4        $\hat{y}^{(k)} =$  (i) _____
5       if  $\hat{y}^{(k)} \neq y^{(i)}$ :
6          $\alpha_{i,y^{(i)}} =$  (ii) _____
7          $\alpha_{i,\hat{y}} =$  (iii) _____
8        $k \leftarrow k + 1$ 

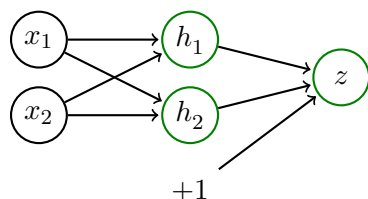
```

- (2 pts) After convergence, how do you classify a new point $\bar{x}$ using only the α 's and the kernel function K ? Write $h(\bar{x})$ explicitly.

6 Neural Networks (6 pts)

Consider a 2-layer feed-forward neural network that takes as input two-dimensional examples $\bar{x} = [x_1, x_2]^T$, and has 1 hidden layer with 2 neurons and ReLU activation.

Input layer Hidden layer Output layer



$$h(\bar{x}; \bar{\theta}) = z = v_1 h_1 + v_2 h_2 + v_0$$

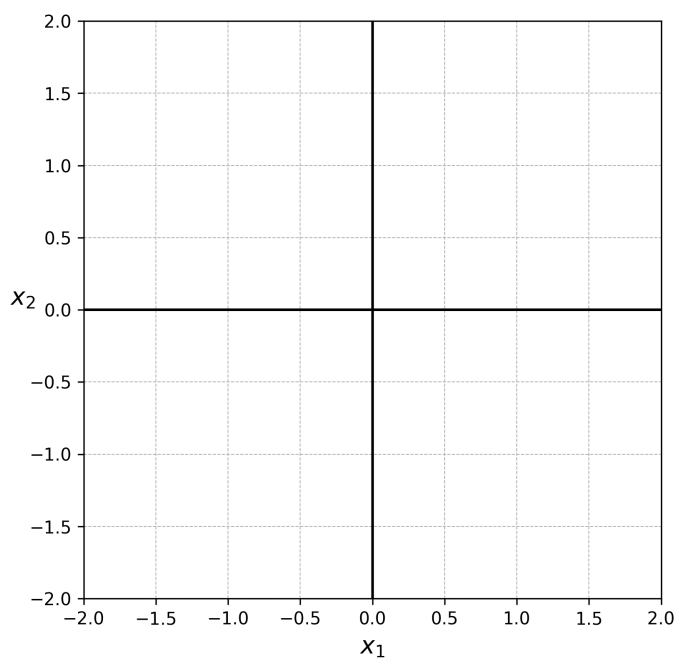
$$z_1 = w_{11}x_1 + w_{21}x_2, \quad h_1 = \max\{0, z_1\}$$

$$z_2 = w_{12}x_1 + w_{22}x_2, \quad h_2 = \max\{0, z_2\}$$

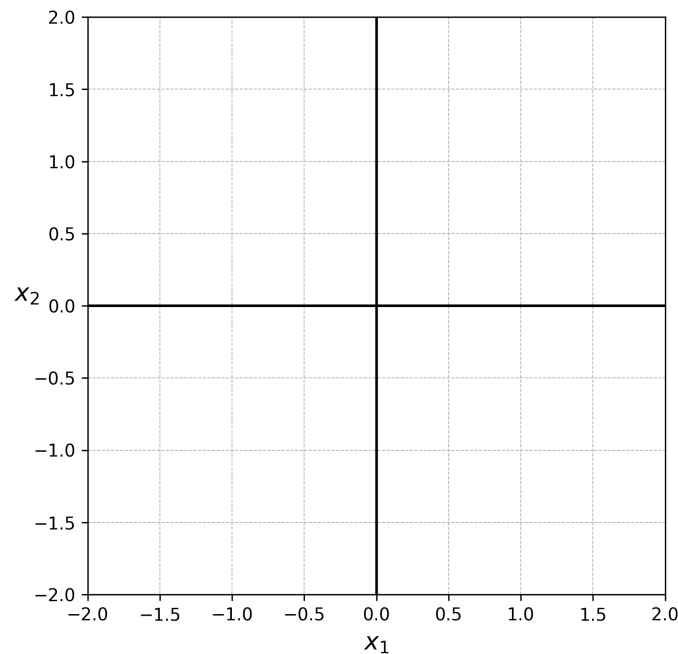
The weights w and v are set as follows.

$$\begin{bmatrix} w_{11} & w_{12} \\ w_{21} & w_{22} \end{bmatrix} = \begin{bmatrix} 1 & -1 \\ 1 & 1 \end{bmatrix}, \quad [v_1 \quad v_2 \quad v_0] = [1 \quad -1 \quad 1]$$

- (a) (2 pts) On the set of axes below, draw the “decision boundaries” corresponding to the two hidden neurons. These are boundaries in the $\bar{x}$ -space where the input to the hidden neurons is exactly zero (i.e., where z_1 equals zero and where z_2 equals zero). Please label the sides of the boundaries with 0 and + to indicate whether the neuron’s output would be exactly zero or positive, respectively.
- (b) (2 pts) On the same set of axes, also draw and label the boundaries in the $\bar{x}$ -space corresponding to $h_1 = \max\{0, z_1\} = 1$ and $h_2 = \max\{0, z_2\} = 1$ where z_1 and z_2 are the inputs resulting from $\bar{x}$.



- (c) (2 pts) On the set of axes below, draw the decision boundary of the network model as a classifier. This is a boundary in the $\bar{x}$ -space where the network output $h(\bar{x}; \bar{\theta})=0$. Be sure to indicate the positive side.



7 Recurrent Neural Network (11 pts)

Recall the binary classification problem in Project 1, where our goal is to predict *in-hospital mortality* for ICU patients given 48 hours of physiological measurements. We can treat this as a sequence classification task and use Recurrent Neural Networks. In particular, you will implement an RNN with LSTM cells. For computational efficiency, we will only consider the first 4,000 examples. We will also only consider the 35 time-varying variables.

You don't need to implement the data generation part—the course staff team has already implemented `q7_prepare_data.py` provided in the starter code. Here is the overview: In the main function of `q7_prepare_data.py`, we first call `generate_features(df)` on every single patient. This function converts a patient's data into a DataFrame, with columns `[Time, Variable, Value]`. We then call `impute_missing_values(X)` to impute the remaining missing values with population mean, and `standardize_features(X)` to standardize the data to mean 0 and std 1. These functions are very similar to what you did in Project 1. *Note, you need to move the data files from project 1 to `q7_data/files` folder before running `q7_prepare_data.py`.*

- (a) Study the implementation of `generate_features(df)` in `q7_prepare_data.py`. And you need to run `q7_prepare_data.py` manually.

The raw data for each patient are irregularly-sampled time-series data, and we will first resample them to be even. For the 48-hour period, we group the observations into **2-hour** windows so there are 24 time windows. Within each window, we compute the mean value and missing flag for each of the 35 variables. We then use a simple forward imputation approach to estimate *some* missing values using `df.ffill()`. This copies an observed value to future unobserved timesteps. All

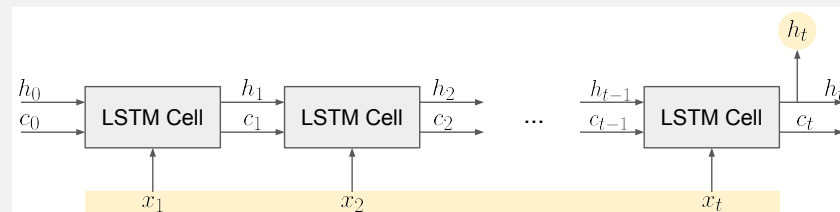
of these preprocessing steps are implemented in `generate_features(df)`. The output of this function is a `pd.DataFrame` with 70 feature columns and 24 rows, one for each 2-hour window. See `sample_features.csv` for an example of the features extracted for one patient in the `q7_data` folder.

- i. (1 pt) Note that even after resampling and forward filling, a patient could still have missing values. When do you think could this happen?
 - ii. (1 pt) Why is it necessary to resample the data, given that we want to apply a standard RNN model (with LSTM cells)?
- (b) (1 pt) Study the RNN architecture below. How many learnable parameters are there? Treat biases associated with input and hidden state as different parameters.
- (c) (2 pts) Complete the RNN model class in `q7_model.py` by implementing the methods `__init__(self, input_size, hidden_size, output_size)`, `init_weights(self)`, `init_hidden(self, N)` and `forward(self, x)`. Note that the forward function will contain a for loop since we are dealing with sequence data.
- (d) (1 pt) In `q7_train.py`, the data are split into train (80%), validation (10%) and test (10%). Run `q7_train.py` to train the model and attach the resulting training plots (for loss and AUROC). At which epoch do we start overfitting? [Note: running `q7_train.py` might take a few minutes. Create a folder called “`q7_checkpoint`” to collect the checkpoints.]
- (e) (1 pt) Run the script `q7_evaluate.py`. Based on your answer to part(d), choose the epoch number to load the checkpoint from. Report the final model performance on the test set (AUROC score).
- (f) Reflection:
 - i. (1 pt) How would the model’s performance and convergence rate be affected if Xavier initialization is replaced with constant initialization? And why? (Hint: You may find <https://docs.pytorch.org/docs/stable/nn.init.html> helpful.)
 - ii. (1 pt) Name at least one more hyperparameter associated with this modeling approach. Then, compare the model’s performance before and after tuning this hyperparameter.
 - iii. (1 pt) Here, we focused on only time-varying data. How might you incorporate the time-invariant data (e.g., age, gender)?
 - iv. (1 pt) What are the differences between an LSTM cell and a GRU cell? If we replaced the LSTM cell with a GRU cell what do you expect would change? (Hint: Implementation is not required to answer this question, though you are encouraged to read about `nn.GRUCell`.)

RNN Architecture

Length of sequence	Input size (per timestep)	Hidden size	Output size
$T = 24$	$d = 70$	$D = 128$	1

1. Input layer: $\bar{x} \in \mathbb{R}^{T \times d}$
2. LSTM layer: (Hint: you need to write a for loop)
 - Please use `nn.LSTMCell`, docs: <https://pytorch.org/docs/stable/nn.html#lstmcell>
 - Hidden state & cell state initializations:
 - $h_0 = [0, 0, \dots, 0]^T \in \mathbb{R}^D$
 - $c_0 = [0, 0, \dots, 0]^T \in \mathbb{R}^D$
 - Parameter initializations
 - Weights: Xavier uniform, for each weight matrix, gain=1
 - Biases: constant 0.0, for each bias parameter
 - Layer output: $h_t \in \mathbb{R}^D$, hidden state at last timestep



3. Output layer: fully-connected layer
 - Parameter initializations
 - Weights: Xavier uniform, gain=1
 - Biases: constant 0.0
 - Activation function: sigmoid
 - Layer output: $\hat{y} \in [0, 1]$

8 Self-Attention (8 pts)

In natural language processing (NLP), self-attention has become a crucial mechanism for capturing dependencies between words in a sentence, regardless of their distance from one another. It allows models like Transformers to effectively attend to all words in a sequence and weigh their importance dynamically when processing a target word.

In this problem, we will explore the concept of self-attention in the context of a simple toy example. The goal is to walk through the self-attention mechanism step-by-step, calculating the key components of this process. You may also find this online tutorial helpful: <https://jalammar.github.io/illustrated-transformer/>.

Consider the input sentence:

“I like ML”

Suppose each word is mapped to a 3-dimensional embedding space, denoted as $\bar{x}_i \in \mathbb{R}^3$, and that the self-attention mechanism generates a query ($Q = [\bar{q}_1, \bar{q}_2, \dots, \bar{q}_N]^T \in \mathbb{R}^{N \times d_q}$), key ($K = [\bar{k}_1, \bar{k}_2, \dots, \bar{k}_N]^T \in \mathbb{R}^{N \times d_k}$), and value ($V = [\bar{v}_1, \bar{v}_2, \dots, \bar{v}_N]^T \in \mathbb{R}^{N \times d_v}$) for each word via parameters $W_q \in \mathbb{R}^{d \times d_q}$, $W_k \in \mathbb{R}^{d \times d_k}$, $W_v \in \mathbb{R}^{d \times d_v}$. **For simplicity, we let $d = d_q = d_k = d_v = 3$.** These are parameters that are learned during training, and these matrices project the embedded inputs to query, key, and value components i.e., $\bar{q}_i = W_q \bar{x}_i$, $\bar{k}_i = W_k \bar{x}_i$, $\bar{v}_i = W_v \bar{x}_i$. In other words, $Q = XW_q$, $K = XW_k$, $V = XW_v$, where $X = [\bar{x}_1, \bar{x}_2, \dots, \bar{x}_N]^T$. The following queries, keys, and values correspond to the words “I”, “like”, and “ML”:

$$\begin{aligned} \text{“I”} \quad \bar{q}_1 &= \begin{bmatrix} 0.3 \\ -0.7 \\ 1.1 \end{bmatrix}, \quad \bar{k}_1 = \begin{bmatrix} 0.2 \\ -0.6 \\ -0.8 \end{bmatrix}, \quad \bar{v}_1 = \begin{bmatrix} 0.8 \\ -1.5 \\ 3.1 \end{bmatrix} \\ \text{“like”} \quad \bar{q}_2 &= \begin{bmatrix} -0.4 \\ 0.5 \\ -2.2 \end{bmatrix}, \quad \bar{k}_2 = \begin{bmatrix} 0.5 \\ -0.5 \\ 0.6 \end{bmatrix}, \quad \bar{v}_2 = \begin{bmatrix} -0.1 \\ 0.6 \\ -0.8 \end{bmatrix} \\ \text{“ML”} \quad \bar{q}_3 &= \begin{bmatrix} 0.1 \\ 0.3 \\ 0.6 \end{bmatrix}, \quad \bar{k}_3 = \begin{bmatrix} -2.1 \\ -1.3 \\ -0.2 \end{bmatrix}, \quad \bar{v}_3 = \begin{bmatrix} -0.9 \\ 1.2 \\ 0.3 \end{bmatrix} \end{aligned}$$

For the following questions, you are encouraged to refer to the provided Python code `q8_transformer.py`, which implements the `TinyTransformer` model. This code will help you calculate the attention scores, apply the softmax function, and compute the final context-aware representations. You should follow the steps outlined below:

- **Calculate Queries, Keys, and Values:** Use the function `compute_qkv(self, X)` in the code to calculate the query ($Q \in \mathbb{R}^{3 \times 3}$), key ($K \in \mathbb{R}^{3 \times 3}$), and value ($V \in \mathbb{R}^{3 \times 3}$) matrices for the sequence using matrix multiplications.

Note that for each word in the sequence, there is a corresponding query vector $\bar{q}_i$, key vector $\bar{k}_i$, and value vector $\bar{v}_i$. To construct the full query, key, and value matrices for the sequence, you concatenate these vectors along the sequence dimension (i.e., stack them vertically for all words in the sentence).

You will need to finish this function by multiplying the input embeddings X with the learned weight matrices W_Q , W_K , and W_V . Compare with the Q , K , V matrices with the provided values above to ensure they are correct. The shape of the resulting matrices should be $[\text{batch_size}, \text{seq_len}, d]$ for Q , K , and V , where seq_len is the number of words in the sentence, and $d = 3$ is the dimensions of the query/key and value vectors. Because we are only processing one sentence at a time, the `batch_size` is 1.

- Calculate Attention Scores: Use the function `attention_scores(self, Q, K)` to compute the attention scores. These scores are the dot products between the query vector of the word of interest and the key vectors of all words. The formula for the attention score is:

$$\text{Attention Score}(\bar{q}_i, \bar{k}_j) = \frac{\bar{q}_i \cdot \bar{k}_j}{\sqrt{d_k}}$$

where $\bar{q}_i$ is the query vector of the word of interest, $\bar{k}_j$ is the key vector of each word, and d_k is the dimension of the query/key vectors (to scale the dot product). This function will return a matrix of attention scores for all words in the input batch.

- Calculate Attention Weights: Apply the softmax function to the attention scores using the function `attention_weights(self, scores)` to normalize them. These normalized scores represent the attention each word gives to all words in the sentence. Complete the function `attention_weights(self, scores)`. To avoid numerical instability, first subtract the maximum value in each row of the attention scores before applying the exponential. Then compute the softmax for each row. The result is a matrix where each row sums to 1, representing the normalized attention weights.
- Compute the Context Vector: Use the function `context_vector(self, weights, V)` to compute the final context-aware representation by taking a weighted sum of the value vectors based on the attention weights. You will need to implement this function by making matrix multiplication on the attention weights and the values.

See further instructions in the code `q8_transformer.py`. You can modify or use the code directly to verify your answers.

- (2 pts) Given the query vector for “ML” $\bar{q}_3$ and all key vectors K , where $K \in \mathbb{R}^{3 \times 3}$, compute the attention scores. Report the attention scores for each word in the sentence above with respect to the query word “ML”. Based on these scores, when processing the word “ML”, which word is most relevant? (*Keep four significant figures*)
- (2 pts) Apply the softmax function to calculate the attention weights. Given these attention weights (i.e., normalized attention scores) and the values, V , what is the context-aware representation $\bar{h} \in \mathbb{R}^3$ for the word “ML”? (*Keep three significant figures*)
- (1 pt) Multi-head attention extends self-attention by using multiple attention heads. Explain how multi-head attention can improve model performance over using a single attention head.
- (1 pt) What is the purpose of the softmax function in a self-attention block?
- (1 pt) Explain the purpose of including $\sqrt{d_k}$ in the attention score calculation. (*Hint: Think about what would happen if the scaling factor is not there.*)
- (1 pt) How can attention capture long-range dependencies in text? (*Hint: Think about why vanilla RNN couldn’t handle long sequence well.*)